

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño

El sistema DashKPI requiere mantener tiempos de respuesta bajos en operaciones que implican múltiples consultas a la base de datos (ej. carga de dashboard, reportes de desempeño y KPIs). Estas consultas generan alta carga sobre el servidor MySQL, afectando la experiencia del usuario y el rendimiento general del backend.

Orientación de la decisión

Implementar un sistema de cache distribuido utilizando Redis como componente intermedio entre el backend (Django) y la base de datos.
El objetivo es almacenar resultados de consultas frecuentes o costosas y así reducir el número de accesos directos a MySQL, mejorando el tiempo de respuesta del sistema.

Supuestos

Las consultas de reportes y KPIs se repiten con frecuencia y pueden almacenarse temporalmente sin riesgo de inconsistencia.

Redis ya está disponible en la infraestructura como servicio de mensajería y puede ampliarse para cache.

Los datos del dashboard no requieren actualización en tiempo real estricto.

Restricciones

El cache no puede comprometer la integridad de los datos; debe invalidarse ante actualizaciones relevantes.

El almacenamiento en Redis debe estar limitado para evitar sobrecarga de memoria.

Se deben definir políticas de expiración (TTL) y coherencia de datos.

Requerimientos relacionados

RNF-03: Desempeño del sistema (tiempo de carga del dashboard < 2 s).

RNF-05: Escalabilidad y eficiencia en el uso de recursos.

RF-04: Consultar y visualizar indicadores actualizados en el dashboard.

Alternativas consideradas

Alternativa 1 – Cache en memoria local del backend: usar el cache interno de Django.
Alternativa 2 – Cache distribuido con Redis: implementar cache global entre instancias del backend.
Alternativa 3 – Optimización SQL con índices y vistas materializadas: reducir tiempos de consulta en la base de datos.

Evaluación de alternativas

La Alternativa 1 es sencilla, pero no escala bien con múltiples instancias del backend.
La Alternativa 3 mejora el rendimiento de las consultas, pero no reduce la carga general sobre el servidor ni el número de accesos.
La Alternativa 2 (Redis distribuido) ofrece el mejor equilibrio entre rendimiento, escalabilidad y facilidad de integración, además de aprovechar infraestructura ya existente.

**Alternativa
seleccionada y
Justificación**

Se selecciona la Alternativa 2 – Cache distribuido con Redis, por ofrecer bajo tiempo de respuesta, reducción de carga en MySQL y compatibilidad con la arquitectura distribuida del sistema.
Permite mantener una alta disponibilidad sin duplicar consultas costosas y mejora la experiencia del usuario final.

**Decisiones
relacionadas**

Ajustar TTL de cache según frecuencia de actualización de KPIs.
Implementar invalidación automática de cache tras operaciones de escritura en el backend.
Registrar métricas de aciertos (hit rate) para evaluar su efectividad.

Comentarios

Esta decisión complementa la arquitectura de despliegue, fortaleciendo la capa de rendimiento y asegurando una experiencia fluida al usuario incluso bajo alta concurrencia.